AIDA vs Aider

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-06-09

The TL;DR: **these are different layers, and they compose.** Aider is a terminal pair-programmer that turns a conversation into code and auto-commits every change. AIDA is the spec graph + lifecycle that sits *above* whatever does the editing. Aider doesn't try to be a requirement tracker; AIDA doesn't try to be your editing loop. The honest framing isn't "AIDA vs Aider" — it's "Aider edits, AIDA remembers why."

This page is in the "adjacent neighbor, not direct competitor" category: Aider optimizes for a *different job* than AIDA, and the interesting question is how to run them together.

What Aider is

Aider is a mature, focused, terminal-based AI pair-programming tool. You talk to it; it edits your files and **commits each change to git automatically** with an LLM-generated message. It builds a "codebase map" (an LLM-derived index of your repo) to ground its edits.

Be precise about what it's genuinely good at:

- **Auto-commit-per-turn is the gold standard.** Every change becomes a git commit with a sensible message, with no extra ceremony. This is a genuinely excellent pattern — AIDA's `(SPEC-ID)` trailer convention is more deliberate but expects the agent/operator to write the commit.
- **Zero infrastructure.** A config file and your git repo. Nothing to stand up.
- **Tight, fast edit loop.** It is built to make changes, not to plan programs or track requirements — and it does that one job very well.

What Aider is *not*

- It has **no task or spec model** — it's chat-driven. There's no stable ID for "the feature," no typed relationship between features, no queue.
- Its **commit messages describe the diff**, not the intent — there's no link back to a spec, because there's no spec to link to. Six months later the commit tells you *what changed*, not *why it was supposed to exist*.

- It's **single-agent**. No coordination layer, no role handoffs, no lifecycle.

None of that is a flaw — it's scope. Aider is deliberately a pair-programmer, not a project memory.

Where Aider holds up (and you should just use it)

- You want the fastest path from "change this" to a committed diff.
- The unit of work is a conversation, not a tracked feature graph.
- You're solo or one-off, and the durable "why" doesn't need to outlive the session. (See when *not* to use AIDA — this is one of those cases.)

If that's your shape, **use Aider.** AIDA's graph wouldn't earn its keep.

Where AIDA adds a layer Aider doesn't have

The moment you want the work to be *remembered as intent* — "what implements the auth epic?", "why does this function exist?", "what's blocked on what?" — you're asking for a layer Aider doesn't model: stable spec IDs, typed relationships, code↔spec traces, and a lifecycle that survives the session. That's AIDA.

How they compose

AIDA is **the coordination + memory + lifecycle layer above whichever single-agent tool does the editing.** `aida queue work` invokes `claude` today, but the pattern is tool-agnostic — the same scaffold can drive `aider` as the implementer. The division of labor:

- **AIDA** holds the spec, routes it to a queue, and records the linkage.
- **Aider** does the actual editing turn and auto-commits.
- The one seam to mind: Aider writes its *own* commit message, so to keep AIDA's auto-bump working you'd append the active `(SPEC-ID)` trailer to Aider's commit (or let AIDA's commit convention wrap the turn). The substrate already knows the active scope; adding the trailer is mechanical.

  Worth borrowing: Aider proves auto-commit-per-turn works *without* losing operator control. An `aida queue work --auto-commit` that generates the message from the diff and appends the `(SPEC-ID)` trailer is a tracked opportunity — Aider is the proof the model is sound.

Bottom line

Aider is a great editing loop. AIDA is the memory and lifecycle around editing loops. Use Aider to make changes fast; reach for AIDA when you need the changes to stay tied to *why* — and run Aider *inside* AIDA when you want both.

See also

- composition.md — the general “use AIDA *with* an editor” recipe.
- when-not-to-use-aida.md — when Aider alone is enough.
- vs-continue.md — the other CI-native editor neighbor.